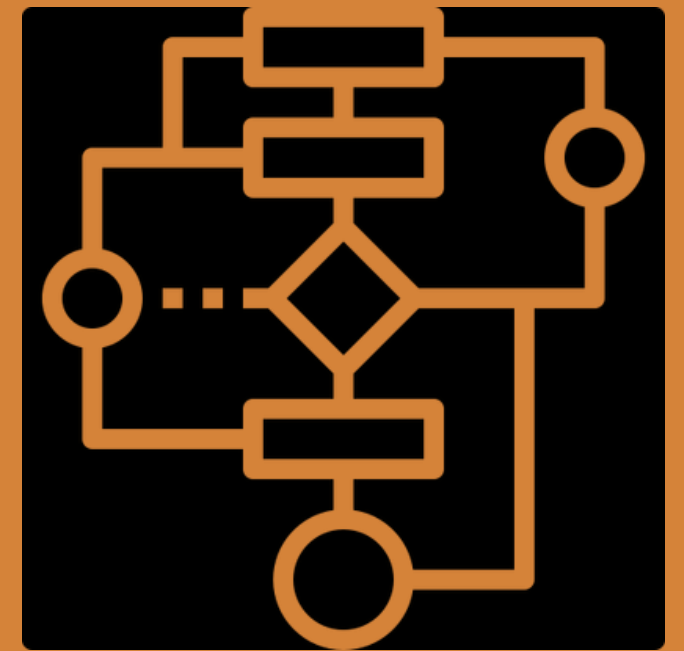
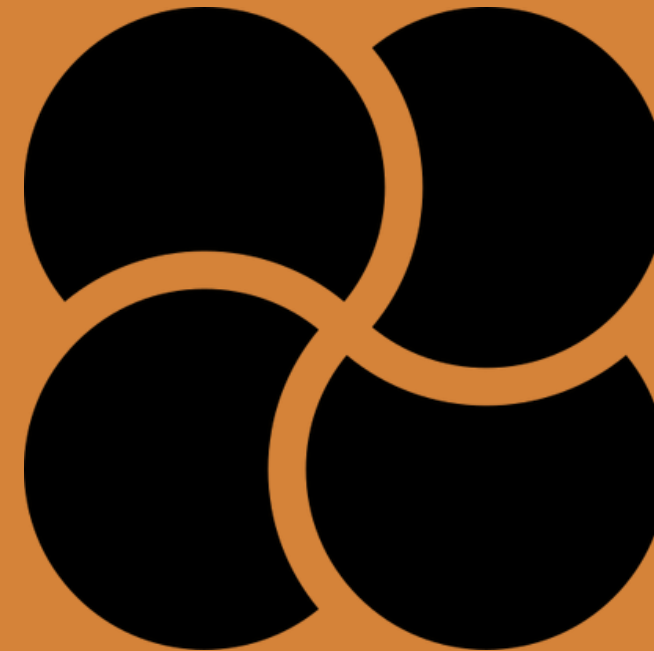
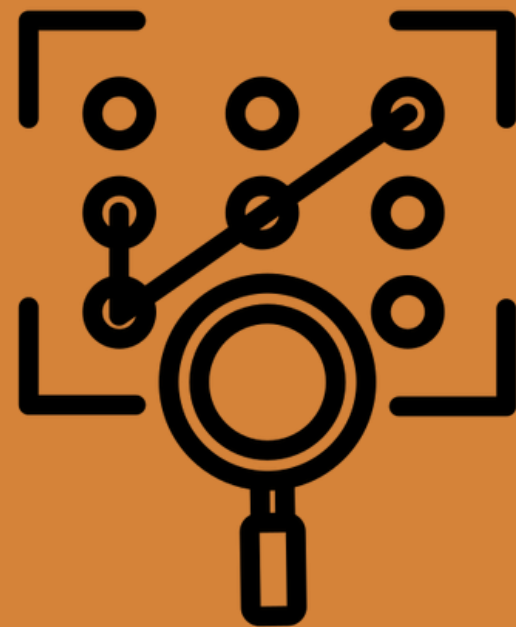
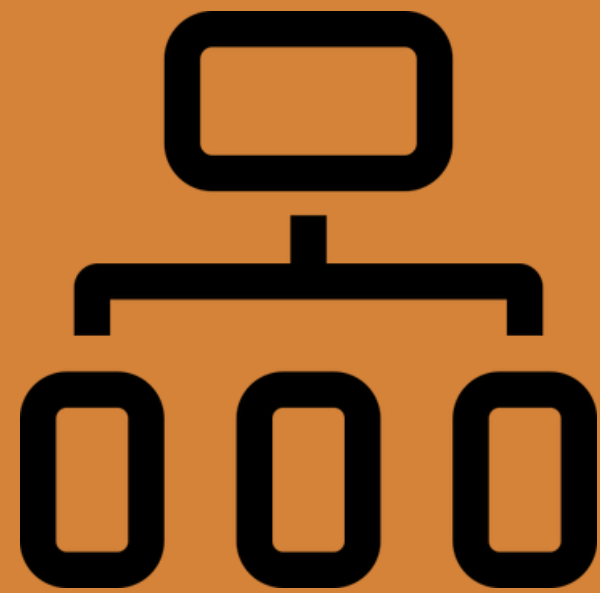


Pensamiento Computacional para Comunicaciones

2026-II



Luisa Gomez

luisa.gomez@pucp.edu.pe

<https://github.com/4591526>





¿Colab, VS Code o Jupyter?



Herramientas populares para programar en Python, cada una con sus propias fortalezas.

	Google Colab	Visual Studio Code	Jupyter Notebook
 ¿Qué es?	 Entorno de notebooks en la nube de Google, ideal para aprendizaje, análisis de datos y colaboración.	 Editor de código ligero y potente, que se instala en tu computadora y es altamente personalizable.	 Entorno de notebooks abierto y flexible, ideal para análisis de datos, exploración y documentación.
 Dónde se usa	 En la nube (navegador web)	 En tu computadora (Windows, macOS, Linux)	 En tu computadora (o en servidores)
 Instalación	 No requiere instalación	 Requiere instalación	 Requiere instalación (Anaconda recomendado)
 Lenguajes	 Principalmente Python	 Soporta muchos lenguajes (Python, JavaScript, C++, etc.)	 Principalmente Python (con kernels para otros lenguajes)
 Colaboración	 Muy fácil (compartir enlaces, trabajo en tiempo real)	 Extensiones para colaboración (Live Share), pero no nativo	 Se puede compartir el notebook, pero sin edición en tiempo real
 Extensiones / Personalización	 Limitadas	 Extensiones ilimitadas (muy personalizable)	 Extensiones a través de plugins y nbextensions
 Requiere recursos locales	 No (usa recursos de Google como CPU, GPU, TPU)	 Sí (depende de tu computadora)	 Sí (depende de tu computadora o servidor)
 Ideal para	 Estudiantes, ciencia de datos, colaboración rápida, prototipado en la nube	 Desarrollo de software, proyectos grandes, trabajo local profesional	 Análisis de datos, investigación, visualización, reportes y notebooks interactivos

Notebooks de Python

Algunos consejos para empezar:

- Las celdas de un notebook contienen código o texto. Si ejecutas cualquier celda, se ejecutará el código o se renderizará el texto.
- **Formas de ejecutar una celda:**
 - a) **Haz clic en el botón de Run**  en la parte izquierda del bloque de código.
 - b) **Alt + Enter** ejecuta la celda actual y crea una nueva celda.
 - c) **Ctrl + Enter** ejecuta la celda actual sin crear una nueva celda.

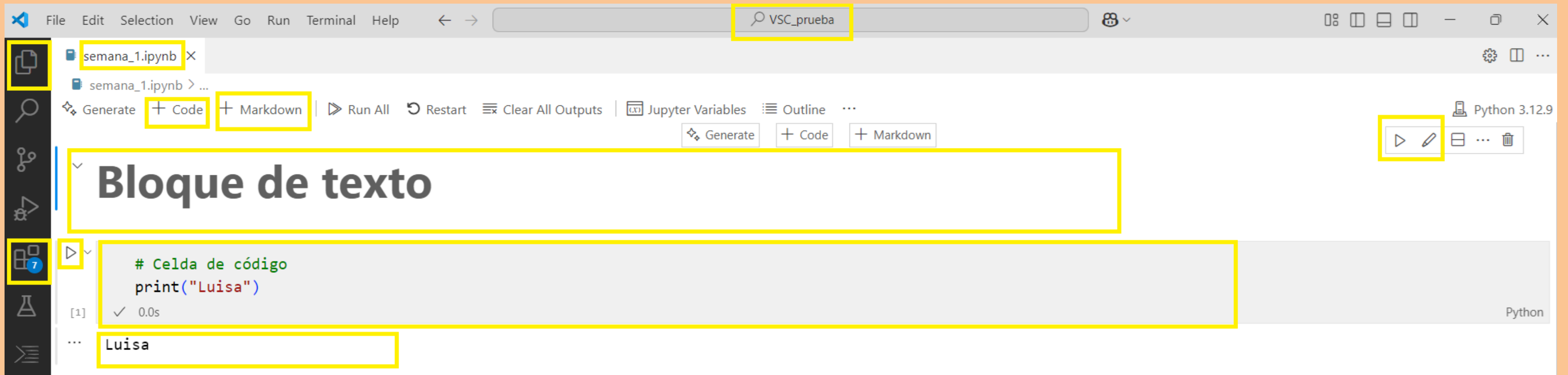
Consejo: Mantén ordenado tu Notebook. Si ejecutas las celdas de manera desordenada puede que obtengas resultados que no esperas.

Google Colab(oratory)

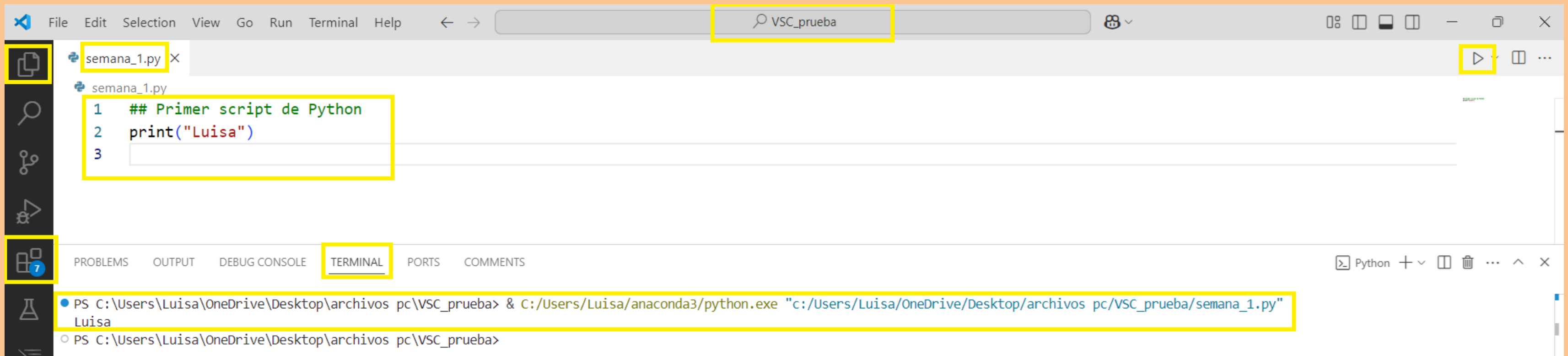


Nota: Si tienes una Mac te recomiendo usar Chrome y no Safari para poder usar Colab sin problemas.

Jupyter Notebook



Visual Studio Code



Nota: Revisa la clase Introducción en el siguiente enlace
<https://pcc-interactivo.streamlit.app>

print() en Python: ¿Qué es y cómo usarlo?



La función `print()` muestra información en la pantalla (salida).

¿Qué hace `print()`?



`print()` envía uno o más argumentos a la salida estándar (la pantalla).

Estructura (Sintaxis)

```
print( argumento1, argumento2, ..., argumentoN )
```

Nombre de
la función

Argumento 1
(puede ser texto,
número, variable, etc.)

Argumento 2

Argumento N
(opcional)

¿Qué son los argumentos?



Son los valores o expresiones que se colocan dentro del paréntesis de `print()`. Pueden ser uno, varios o ninguno.

- No te olvides que se debe escribir **print** sin un espacio antes de los paréntesis **()** .
- El mensaje que contiene la función **print()** puede ser cualquier tipo de datos de Python (strings, int, float, bool, listas, entre otros).

Nota: Revisa la clase Mi primer código en Python en el siguiente enlace
<https://pcc-interactivo.streamlit.app>

Argumentos

- Es la información que se puede ingresar en las funciones.
- Estos son específicos luego de llamar a la función dentro del paréntesis.
- Se puede agregar cualquier cantidad de argumentos separados por comas.

```
[ ] print("21/08/2024") # un solo argumento sin espacios
```

```
➞ 21/08/2024
```

```
▶ print("21","/","08","/","2024") # 5 argumentos con espacios
```

```
➞ 21 / 08 / 2024
```

Argumentos

- Recuerda que la separación por defecto entre los argumentos es un espacio vacío.
- Con el parámetro **sep='separador'** (opcional), pueden especificar cómo separar los datos si se trata de más de uno.

```
[ ] print("Hola", "¿Cómo están?", sep="---")
```

```
⇒ Hola---¿Cómo están?
```

```
[ ] print("Hola", "¿Cómo están?", sep=";")
```

```
⇒ Hola;¿Cómo están?
```

```
[ ] print("Hola", "¿Cómo están?", sep=":")
```

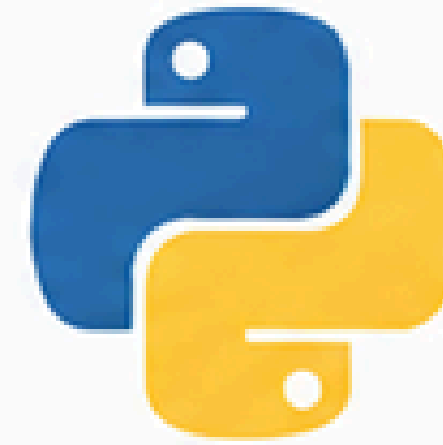
```
⇒ Hola:¿Cómo están?
```

Sintaxis básica de Python

Los comentarios se inician con **#** y todo lo que vaya después en la misma línea sería considerado un comentario.

```
1 # Así se escribe un comentario en Python.  
2 # Se usa para ofrecer a otros programadores o  
3 # a nuestro yo del futuro una explicación acerca  
4 # del código que hemos escrito.
```

Variables en Python



edad = 25

Variable

Nombre que identifica un dato en memoria.

Asignación

El operador = asigna un valor a la variable.

Valor

Dato almacenado en la variable.

Variables

- Las variables almacenan datos (str, int, float, bool, list, entre otros).
- Una variable se crea en el momento en que se le asigna un valor por primera vez.
- Los strings en Python siempre van entre comillas simples o dobles.
- No utilices nombres reservados, como **print** o **type** (estos se volverán de otro color en los Notebooks). Aquí hay una lista de palabras reservadas en Python: https://www.w3schools.com/python/python_ref_keywords.asp

Variables

- Los nombres de las variables no van entre comillas.
- Los nombres de las variables deben comenzar con una letra (a - z, A - B) o un guion bajo (_). El resto del nombre de la variable puede consistir en letras, números y guiones bajos.
- Utiliza nombres de variables claros, significativos y descriptivos para que tu código siga siendo comprensible